

Clase 057 — Almacenamiento seguro de contraseñas: bcrypt, scrypt y Argon2

Parte: **2** — **Criptografía aplicada** · Fuente: *Real-World Cryptography* (Wong) y OWASP Password Storage Cheat Sheet 🕒 Duración estimada: **100 min** · Nivel: **Intermedio**

🎯 Objetivo

Aprender a almacenar contraseñas de forma resistente a cracking usando funciones de derivación de clave lentas y con memoria intensiva (bcrypt, scrypt, Argon2), y por qué usar SHA-256 o MD5 para contraseñas es un error grave. El alumno entenderá el papel del salt, el pepper, el factor de coste y estimará el coste real de un ataque con hashcat en un entorno de laboratorio.

⚠️ **Nota ética:** el cracking de hashes se practica **solo** sobre contraseñas y volcados propios de laboratorio. Atacar credenciales ajenas sin autorización es ilegal.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** por qué las contraseñas requieren funciones lentas, no hashes rápidos.
- 2. **Aplicar** salt (único por usuario) y pepper (secreto global) correctamente.
- 3. **Configurar** parámetros de coste de bcrypt/scrypt/Argon2.
- 4. **Almacenar y verificar** contraseñas con Argon2id en Python.
- 5. **Estimar** el coste de un ataque de diccionario/fuerza bruta con hashcat.

📖 Temas

#	Tema	Por qué importa
1	Por qué no SHA-256 para contraseñas	Demasiado rápido de crackear
2	Salt (por usuario)	Rompe rainbow tables
3	Pepper (secreto global)	Defensa adicional
4	bcrypt y factor de coste	Estándar histórico
5	scrypt y Argon2 (memory-hard)	Resisten GPU/ASIC
6	Verificación en tiempo constante	Evita timing
7	Ataques con hashcat (lab)	Medir la resistencia real

Definiciones y características

- **Función de derivación de clave (KDF) para contraseñas:** función lenta y ajustable que transforma la contraseña en un hash. Característica: coste configurable para frenar ataques.
- **Salt:** valor aleatorio único por usuario almacenado junto al hash; impide rainbow tables y colisiones entre usuarios con igual contraseña.
- **Pepper:** secreto global no almacenado en la base de datos (en HSM/KMS); añade una capa que el atacante no obtiene con solo el dump.
- **bcrypt:** KDF basada en Blowfish con factor de coste (rondas). Limitada a 72 bytes de entrada.
- **scrypt:** KDF con coste de memoria, dificultando ataques con hardware especializado.
- **Argon2id:** ganador del Password Hashing Competition; recomendado por defecto; combina resistencia a GPU y a side-channels.
- **hashcat:** herramienta de recuperación de contraseñas (para auditar la fortaleza de tus propios hashes).

Herramientas y preparación

```
pip install argon2-cffi bcrypt
# hashcat solo para laboratorio propio
hashcat --version 2>/dev/null || echo "instala hashcat para el lab de auditoría"
```

Usa únicamente hashes y contraseñas de prueba generados por ti.

Laboratorio guiado

1. Hashea y verifica con Argon2id:

```
python from argon2 import PasswordHasher ph = PasswordHasher(time_cost=3, memory_cost=65536,
parallelism=4) h = ph.hash("contraseña-de-prueba") print(h) # incluye salt y parámetros
ph.verify(h, "contraseña-de-prueba") # True o excepción
```

2. bcrypt con factor de coste:

```
python import bcrypt h = bcrypt.hashpw(b"prueba", bcrypt.gensalt(rounds=12))
print(bcrypt.checkpw(b"prueba", h))
```

3. **Compara la lentitud.** Mide el tiempo de hashear con SHA-256 vs Argon2id; observa que Argon2 es órdenes de magnitud más lento (por diseño).
4. **Ataque de diccionario en laboratorio.** Genera unos pocos hashes bcrypt de contraseñas débiles propias y córrelos con hashcat contra una wordlist pequeña; mide cuántas caen y en cuánto tiempo. Repite subiendo el factor de coste y observa el encarecimiento del ataque.
5. **Calibra parámetros.** Ajusta `time_cost` / `memory_cost` para que un hash tarde ~250-500 ms en tu hardware de producción objetivo.

Ejercicios

1. Explica por qué un salt único por usuario derrota las rainbow tables.

2. ¿Dónde debe guardarse el pepper y por qué no en la misma base de datos?
3. Compara el tiempo de cracking de SHA-256 vs Argon2id para la misma wordlist.
4. Calibra Argon2id para ~300 ms por hash en tu máquina.
5. Investiga por qué bcrypt trunca a 72 bytes y cómo afecta a contraseñas largas.
6. Diseña una política de migración de hashes SHA-256 heredados a Argon2id.

Reto verificable

Implementa un módulo de registro/login que almacene contraseñas con Argon2id (salt automático, parámetros calibrados) y verifique en tiempo constante, más un migrador que rehashee credenciales antiguas al iniciar sesión. **Criterio de aceptación:** dos usuarios con la misma contraseña tienen hashes distintos, la verificación acepta la correcta y rechaza la incorrecta, y las cuentas legacy se actualizan transparentemente.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Contraseñas con SHA-256/MD5	Demasiado rápidas; migra a Argon2id/bcrypt/scrypt
Salt reutilizado o global	Debilita; usa salt aleatorio por usuario
Factor de coste demasiado bajo	Fácil de crackear; calíbralo a cientos de ms
Comparar hashes con <code>==</code>	Timing; usa la verificación de la librería
bcrypt con contraseña >72 bytes	Se trunca; considera pre-hash o Argon2

? Preguntas frecuentes

- ? **¿Cuál elijo: bcrypt, scrypt o Argon2?** Argon2id por defecto en diseños nuevos. bcrypt sigue siendo aceptable; scrypt es válido donde ya se usa.
- ? **¿El salt debe ser secreto?** No; el salt puede ser público (se guarda con el hash). Lo secreto es el pepper.
- ? **¿Puedo "cifrar" contraseñas en vez de hashearlas?** No; hashea con una KDF. Cifrar implica poder descifrar, un riesgo innecesario para autenticación.

Referencias

- OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- RFC 9106 (Argon2) — <https://www.rfc-editor.org/rfc/rfc9106>
- Wong, *Real-World Cryptography*, cap. 8.
- hashcat — <https://hashcat.net/hashcat/>

Siguiente clase

Clase 058 - Generación de aleatoriedad segura (CSPRNG)